

THE BLUEPRINT & THE TOY

Intro to Classes & Objects

Python Programming | Grade 4

Imagine This...

Imagine if you had a big pile of colorful LEGO blocks but absolutely no instructions. [cite: 186]

Visual: A pile of bright LEGO blocks next to a missing manual. [cite: 184]

How would you know exactly what to build? Is it a car? A castle? A robot? [cite: 186]

In Python, a **Class** is just like that instruction manual! It tells the computer exactly how to build something step-by-step. [cite: 187]

Without the manual (Class), the blocks are just blocks. With the manual, you can build anything! [cite: 187]

What is a Class?

A **Class** is a blueprint or a plan. Think of it like a **Cookie Cutter** used for making sweet dumplings (like Gujiya or Modak)! [cite: 194]

Visual: An Indian sweet dumpling cookie cutter. [cite: 192]

- **Blueprint:** A plan or a shape. [cite: 196]
- **Template:** Helps you make many things that look exactly the same. [cite: 196]

The cutter itself isn't the sweet you eat—it's just the *plan* for how the sweet should look! [cite: 194]

What is an Object?

An **Object** is the actual thing made from the blueprint. [cite: 200]

Think of the **Maruti Suzuki Swift** car parked in your society. The design at the factory is the Class, but the real car you can see and touch is the Object! [cite: 200]

Visual: A Maruti Suzuki Swift car parked on a street. [cite: 198]

- Objects are created using the blueprint (Class). [cite: 201]
- Each object is a real thing you can see and use. [cite: 201]

Has vs. Does

Every object in the world has special details and can do special things. In coding, we call these **Attributes** and **Methods**. [cite: 216]

Attributes are what an object **HAS**. [cite: 217]

Example: A Gujiya (Object) has a *Shape* and a *Size* (Small or Big). [cite: 205]

Methods are what an object **DOES**. [cite: 217]

Example: For our cookie, we have a Bake () method that makes it ready to eat! [cite: 206]

Visual: A Hero Motocorp Bike factory where designs (Classes) become bikes (Objects). [cite: 210]

Let's Code a Student Class!

We are going to create a blueprint called Student so Python knows how to make student objects. [cite: 219, 220]

Step 1: The Blueprint

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def introduce_self(self):
        print("Hi, I am", self.name)
```

Step 2: Creating an Object

Now, let's create a real student named **Kabir**! [cite: 225]

```
kabir = Student("Kabir", 4)
kabir.introduce_self()
```

Kabir is an Object. He has attributes (name="Kabir", grade=4) and can do actions like `introduce_self()`! [cite: 227]

Why do we need Classes?

Classes help us keep our code neat, just like a **tidy cupboard!** [cite: 230, 232]

- Organize code logically. [cite: 230]
- Make many objects easily from just one blueprint. [cite: 230]

The Blueprint Challenge!

Test your knowledge: [cite: 237, 238]

1. Which is the Class: **Cake Recipe** or **Chocolate Cake**?
2. Name 3 attributes of a **School Bus** object.
3. What does a **Remote Control** do? Give one example method.
4. True or False: An **Object** is a blueprint.

Great job, Programmer! You've mastered the basics of Classes and Objects! [cite: 236]